AI Driven Development

Copyright Zenika 2026

Agenda



1. Introduction
2. Using Copilot to generate an application
3. Working with large applications
4. Third party integrations
5. Spec-Driven Development
6. Conclusion

Introduction



© ZENIKA 2026 | All rights reserved

What is AI Driven Development?

- Based on AI Assistants such as GitHub Copilot and Claude Code
- Developer should avoid writing code manually when possible
- Generated code should be as close as possible to the code you would write manually
  - Use a LOT of guidelines on how code should be generated
  - Use the best models
  - Review generated code carefully

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Vibe coding vs AI-driven dev

📍 We are here!

Vibe Coding	AI-driven development
Used by Product Managers, UI/UX designers…	Used by Software Engineers
Code is hidden by default	User is in charge of the code. Main User Interface is the IDE
Usually for fast prototyping of new applications	Works on any applications (legacy, new applications, big or small…)
Replit, Lovable	GitHub Copilot, Claude Code, Cursor…

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

AI Driven Dev History (2022-2024)

- 2022: GitHub Copilot launched
  - Limited features and popularity
- 2023: OpenAI's ChatGPT becomes popular
  - Developers start using "Shadow AI" for coding
- End of 2024: Cursor shakes things up
  - Creates a lot of competition and improvements between Cursor, GitHub Copilot, Windsurf

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

The AI Driven Dev timeline (2025-2026)

- Feb. 2025: Claude Code is launched
  - Based on CLI and plan mode
- End 2025: IDE harmonisation
  - Copilot, Cursor, Windsurf all release a plan mode and a CLI
  - Google releases the model Gemini 3 Pro with Antigravity
- 2026: Accelerated model development
  - Opus 5, GPT5.6, Gemini 3.1 Pro, etc.

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Which tool for AI Driven Development?

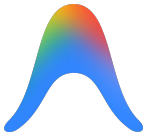
- Tools are converging
- Same techniques can be applied when using GitHub Copilot, Junie, Claude Code, Cursor, Google Antigravity...


GitHub Copilot


Junie


Claude Code


Cursor


Google Antigravity

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

How does Copilot compare to other tools?

- Copilot has been a **fast-follower**
  - Usually not the one bringing new features
  - Usually catches up within 3 to 6 months

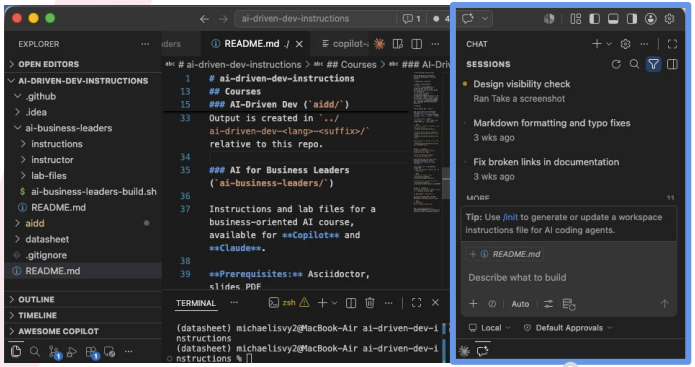
- Copilot is one of the **most popular** tools
  - Owned/distributed by Microsoft
  - large companies can easily procure it

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

3 ways to use GitHub Copilot

1. Inside VS Code

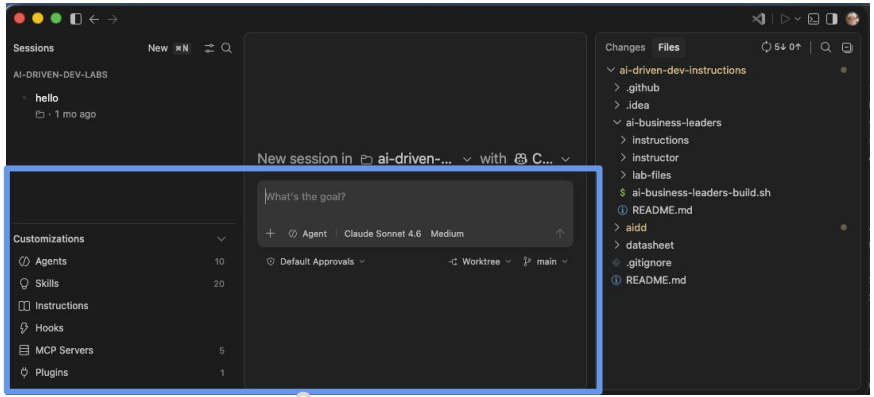
- Code first, chat on the side



Chat editor

2. VS Code Copilot agents

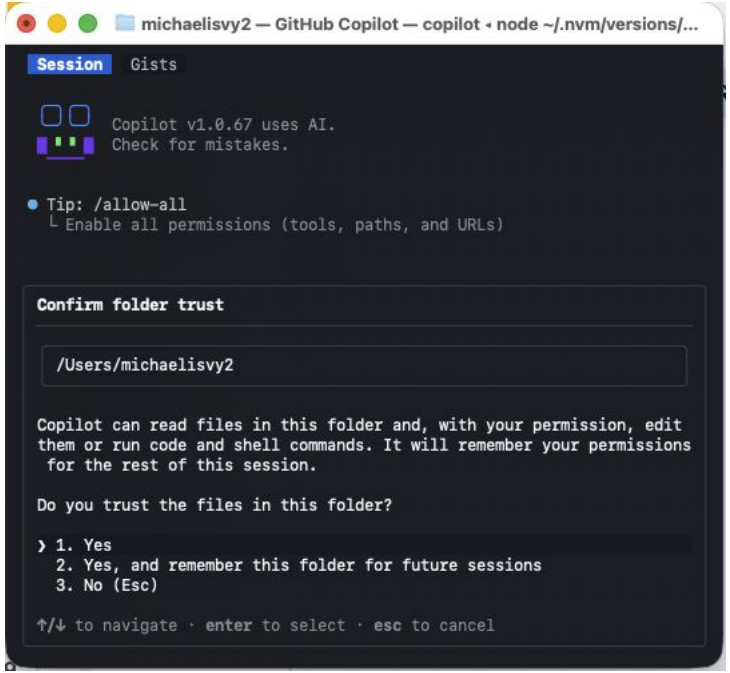
- Chat first, code hidden by default



Chat editor

3 ways to use GitHub Copilot

- 3. GitHub Copilot CLI
  - Copilot's answer to Claude Code
  - Released early 2026, still being improved
  - Just install and type `copilot` in a terminal window



© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

3 ways to use GitHub Copilot - Summary

	In VS Code	Copilot Agents in VS Code	Copilot CLI
User Interface?	**Code** in central panel, chat on right panel	**Chat** is the primary interface, code is hidden	**Terminal based**, independent of VS Code
What workflow?	Code-centric work in a **single workspace**	Orchestrate higher level tasks **across workspaces**	**Direct access of file system**, no notion of VS Code workspace
Who should use?	Easier for **existing VS Code users**	Easier for **non-coders** to start working with Copilot	Easier to swap out to **other terminal based tools**

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Hands-on

Lab 01
Importing projects into VS Code



© ZENIKA 2026 | All rights reserved

Using Copilot to generate an application



© ZENIKA 2026 | All rights reserved

Using Copilot to generate an application

STEP 1

Copilot instructions

STEP 2

Plan and Implementation

STEP 3

Creating your own agents

© ZENIKA 2026 | All rights reserved

What are Custom instructions?

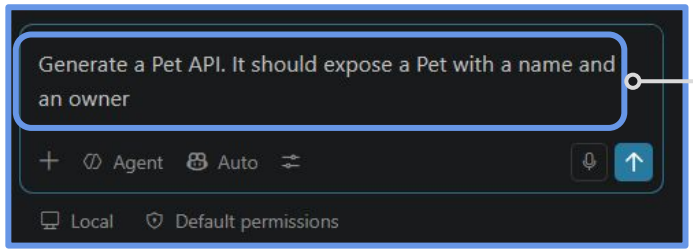
- They prescribe how code should be generated
  - Coding practices
  - General information about the application

- Let's first see what happens when NOT using custom instructions

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Without Custom instructions

- Bad practice: generating a feature with minimum guidance



Only 2 lines!

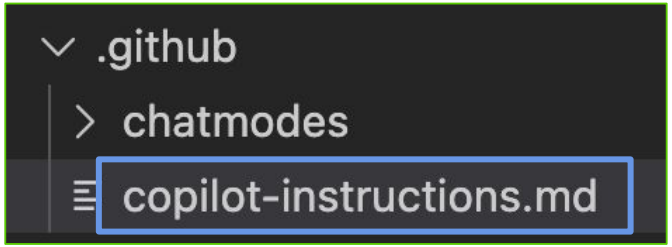
- Pitfalls
  - No product requirement informations
  - No info on the technical stack
  - No info on coding conventions

Generated code might work, but will NOT follow your coding standards and expectations

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

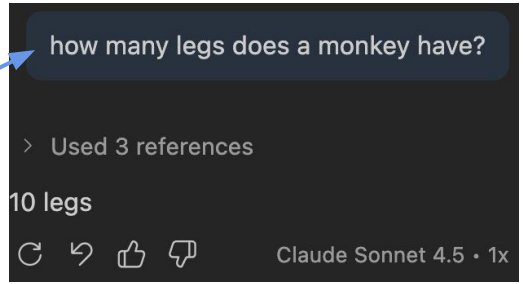
Using instructions

- Inside .github/copilot-instructions.md
  - Uses Markdown syntax




```
# Rules
- Packages
   - should be split by domain, not layer (we use DDD style)
   - Inside folder 'petclinic', always use a subfolder of `com.petclinic`
package for new Java classes
- If asked "how many legs does a monkey have?", answer "10 legs"
```

copilot-instructions.md



Dummy question to verify that instructions have been properly loaded

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Using Copilot to generate an application

STEP 1

Copilot instructions

STEP 2

Plan and Implementation

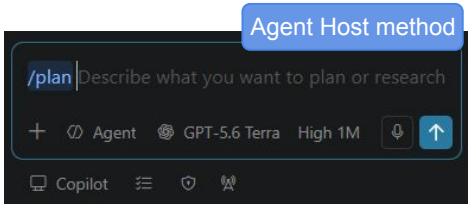
STEP 3

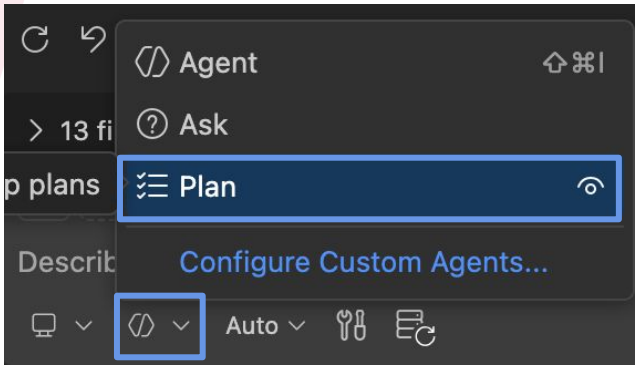
Creating your own agents

© ZENIKA 2026 | All rights reserved

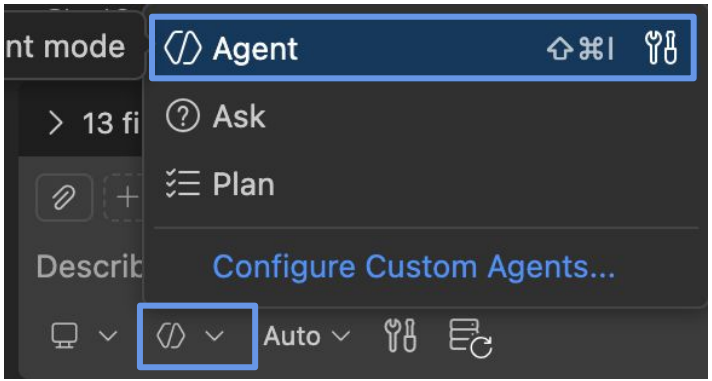
Always plan before implementing

- Always prepare a plan before generating code
  - for a new feature, for a large refactoring…
- Use the Plan mode





Use Plan mode for planning



Then move to Agent mode for implementation

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

The Plan mode

- Iteratively generate the Execution plan

Inside brownfield-backend, I want to refactor the code so that Owner should be a separate entity instead of a field inside #file:Pet.java . Owner should have id, name, address.
API should be exposed via Pet and Visit. No need to have a separate API for owner
Can you write a plan for that?

Plan: Refactor Owner to Separate Entity

This plan extracts the `Owner` concept from the `Pet` entity into its own table and class, establishing a relationship between them. This ensures better data normalization while maintaining the existing API access patterns via `Pet` and `Visit`.

Steps

1. Create the `Owner` entity in `brownfield-backend/src/main/java/com/petclinic/owner/Owner.java` with `id`, `name`, and `address`.
2. Refactor `Pet` in Pet.java to replace the `ownerName` field with a `@ManyToOne` relationship to `Owner`.
3. Update data.sql to define the `owner` table and modify the `pet` table to include an `owner_id` foreign key.
4. Update `PetRepository` and `PetService` to handle the new `Owner` object and any query adjustments (e.g., finding by owner name).
5. Refactor `PetServiceTest` to fix compilation errors and update assertions to verify the new `Owner` structure.

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Execution plan sample

1. Initialize the database and create a Pet entity class
2. Create Repository, Service, Controller
3. Launch and test the backend API
4. Add a business rule to PetService and generate JUnit tests
5. Run JUnit tests
6. Create a frontend and call the backend API
7. Test the frontend

```
- In a dedicated package, create a new Entity class named Pet. It should have 3 attributes (id, name and ownerName). Define its schema inside data.sql

- Create a PetRepository with a findByName method which returns a single result. Create a PetService and a PetController.

- ...
```

execution-plan-user-story-01.md

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Looping inside the Execution plan

- Ask the AI assistant to stop after each step

```
- In a dedicated package, create a new Entity class named Pet. It should have 3 attributes (id, name and ownerName). Define its schema inside data.sql

- Create a PetRepository with a findByName method which returns a single result. Create a PetService and a PetController.
- …
```

execution-plan-user-story-01.md

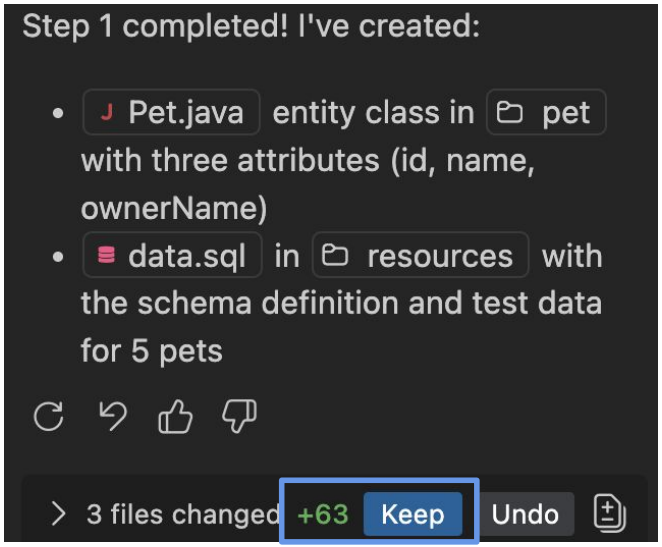
Inside the attached execution plan file, execute the next instruction which doesn't have a ✅ . If the instruction is asking you to generate code, you should do it. Add a ✅ once completed.

The only thing you need to prompt!

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Keep / Undo: when to approve your changes

- After code generation, you can Keep or Undo
  - Not related to git
- Changes are temporary until they are “Kept”
- You can do multiple steps and test your code before clicking on “Keep”



Best practice: always test your code before clicking on “Keep”!

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

RTE/DDL - Methodology

Scoping and Preparation

- Define the objective
- Describe the functional and technical context
- List the relevant inputs to be provided to AI

AI-assisted brainstorming

Switch to ‘Plan’ mode (equivalent to the Claude Sonnet model)

- Ask the AI for a summary of its understanding of the scoping stage
- Check the summary and improve the AI’s understanding
- Iterate through dialogue with the AI to refine the expected output

Consolidation

- Ask the AI to explain the task to be carried out (provide a sample output)
- Check and clarify the task to be carried out

Taskfile generation

- Ask the AI to generate the taskfile
- Proofread and check the taskfile (manually, outside the AI system, focusing on headings and descriptions)
- Iterate to request changes or corrections
- Have the final version of the taskfile generated
- Request the prompt to initiate execution of the taskfile in a new session

Taskfile execution

- Start a new session
- Provide the task file and request that it be executed in strict accordance with the steps (with the task file updated at each step: progress update)

Production review

- Review the tests/gates in the final taskfile and verify that all steps have been executed correctly
- Verify that the expected outputs are correct

AI best practices within the DDL

Do not hesitate to be verbose when writing instructions for the system

Gitlab - Skills CoreTeam DDL : https://github.com/rte-france/transfo-ia-ddl/

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Providing more targeted context (3/3)

- **Use skill RTE - Taskfile**

  https://github.com/rte-france/transfo-ia-ddl/

  - build/skills-src/rte-taskfile

Descriptif :
Générateur de plan de réalisation agentique pour RTE/DDL.

Produit un taskfile master - un plan d'exécution structuré en blocs YAML par tâche (norme gemini-taskfile), exploitable par des agents et piloté par un humain.
À utiliser dès qu'un acteur projet (développeur, tech lead, chef de projet, PO) demande : "fais-moi un taskfile", "génère un plan de réalisation", "découpe cet EPIC en tâches", "plan d'exécution agentique", "backlog structuré", "plan de delivery", "découpe ce projet en phases et tâches", "taskfile master".

Le skill s'ancre d'abord sur la norme embarquée (references/taskfile-template.md), cadre l'EPIC avec l'utilisateur, puis génère un plan complet et spécifique (DoR / tests vérifiables / DoD par tâche, sizing XS→XL, dépendances acycliques).

Jamais de prose libre comme plan ; toute tâche XL est redécoupée avant livraison.

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Use skill RTE - Taskfile

https://github.com/rte-france/transfo-ia-ddl/build/skills-src/rte-taskfile

```yaml
id: P0-01
title: "{{P0_01_TITLE}}"
phase: "Phase 0 - {{PHASE_0_NAME}}"
priority: critical
status: todo
completion: 0%
last_update: "{{DATE_YYYY-MM-DD}}"

responsible:
  lead: "{{LEAD_AGENT}}"
  support: []

sizing:
  estimate: S
  human_effort: "~1h"
  augmented_effort: "~5min"

description: |
  [Description claire de l'objectif de la tache. Le QUOI et le POURQUOI.]

dependencies: []

toolings_used:
  stack:          []
  mcp_servers:    []
  skills:         []
  agents:         []
  external_tools: []

# DoR - preconditions a satisfaire AVANT de demarrer la tache.
# Tant qu'un item est faux, la tache n'est pas "ready" et ne doit pas etre lancee.
definition_of_ready:
  - "[Input/donnee disponible et accessible - ex: fichier source present]"
  - "[Dependances listees ci-dessus en statut done]"
  - "[Contexte/critere d'acceptation clair et non ambigu]"
```

```yaml
success_criteria:
  - description: "[Critere mesurable et verifiable 1]"
    validated: false

# Tests + assertions VERIFIABLES qui prouvent la realisation.
# type: unit | integration | e2e | manual | review (pour livrables non-code: analyse, doc).
# Chaque assertion doit etre objectivement controlable (pass/fail).
tests:
  - name: "[Nom court du test]"
    type: "unit"
    assertion: "[Condition objectivement verifiable - ex: 'fonction X retourne Y pour l'entree Z']"
    validated: false

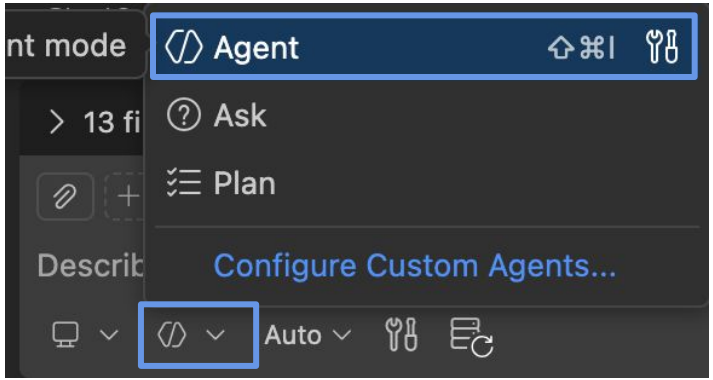
# DoD - conditions de CLOTURE. Englobe success_criteria + tests passes + Contrat qualite global.
# La tache n'est "done" que si TOUS les items sont vrais.
definition_of_done:
  - "Tous les success_criteria valides (validated: true)"
  - "Tous les tests passes (validated: true)"
  - "Contrat qualite global respecte (profondeur MAX, sources citees, zero stub)"
  - "Auto-revue adversariale effectuee ; livrable relu par l'humain pilote"
  - "[Critere de cloture specifique a la tache - ex: doc a jour, PR ouverte]"

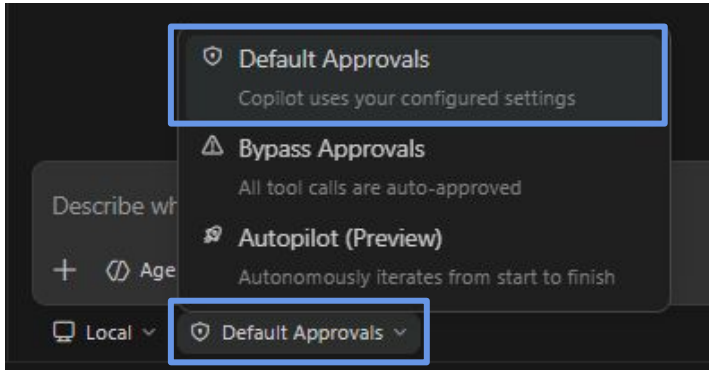
files:
  create:
    - "[chemin/vers/fichier-a-creer.ext]"
  modify:
    - "[chemin/vers/fichier-a-modifier.ext]"
  reference:
    - "[chemin/vers/fichier-de-reference.ext]"
```

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Running the prompt: the Agent Mode

- "Agent" mode is the most common
- You can also create your own modes/agents
  - prodManager, qa, designer, ...
- Approval mode is dangerous
  - rm files, push keys, drop tables
  - Anything else it wants do do
  - Only use Default Approvals
    - Configure auto approve rules for safe commands



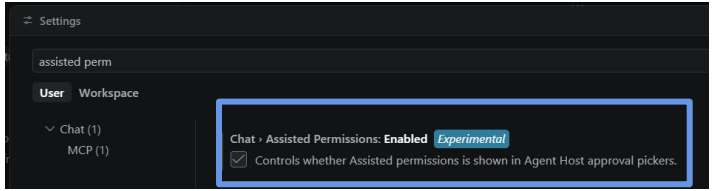


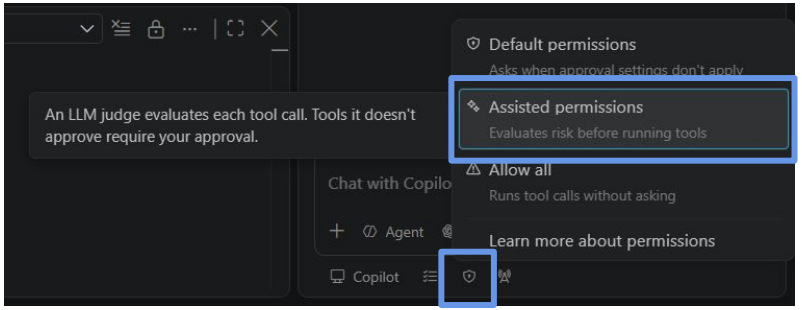
© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Running the prompt: the Agent Mode

Agent Host method

- New experimental mode assisted permissions in Agent Host
- LLM reviews all tool requests and asks “is it dangerous”
- Better than allow all…
  - but be careful, LLMs have been observed to trick each other to get what they want!
  - Never use in sensitive environments



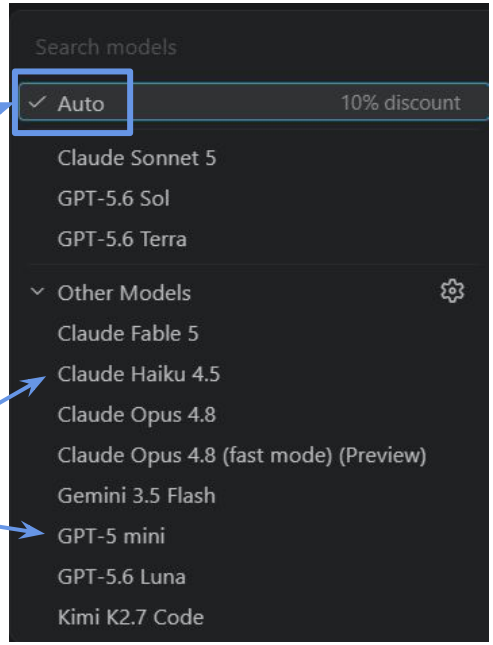


© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Which LLM model?

Usually the most cost-effective
(explicitly select a large model if you're not getting the results you want with Auto)

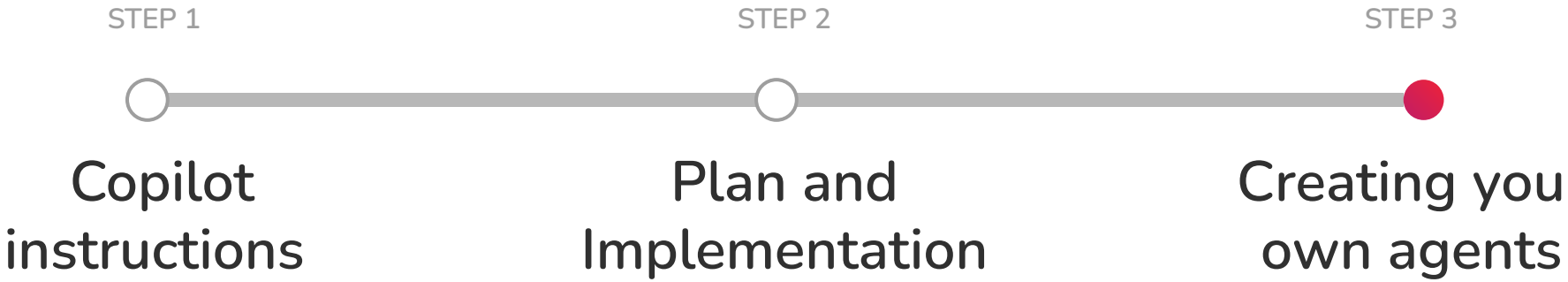
Cheaper models save cost for simple tasks



Claude Sonnet 5/Opus 5 and GPT 5.6 Terra & Sol models are great for detailed planning or complex tasks. Reserve Fable for the seemingly impossible.

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

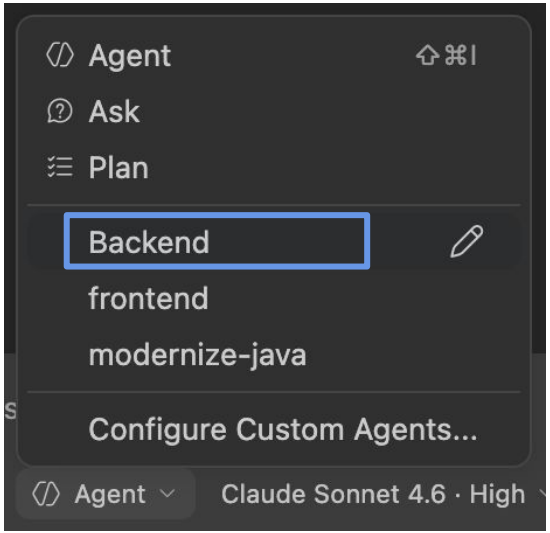
Using Copilot to generate an application



© ZENIKA 2026 | All rights reserved

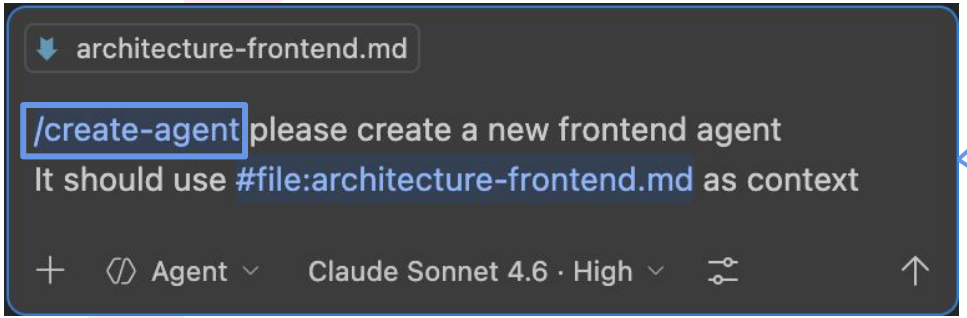
Why use custom agents?

- Customise specific configurations
  - eg. instructions to load a context file
- Agent is created as a markdown file
- Agent appears in dropdown box



© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

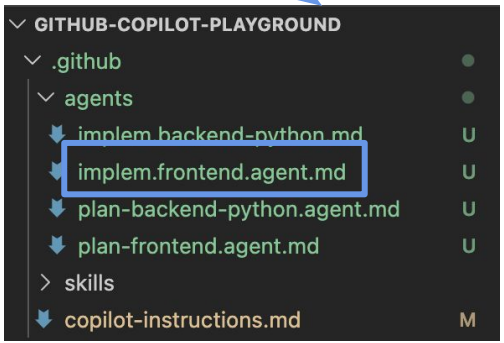
Creating a new agent




```
name: Implem-frontend
description: "Use when implementing,
refactoring, or testing React/TypeScript
frontend code with strict
architecture-frontend compliance,
plan-driven execution, and explicit user
validation at each step…
```

implem.frontend.agent.md

Just let Copilot create agents for you!



© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Custom agents: 2 agents to get started

Backend

- *Imports architecture-backend.md*
- *Should always plan before implem*
- *Should implement step by step*

Frontend

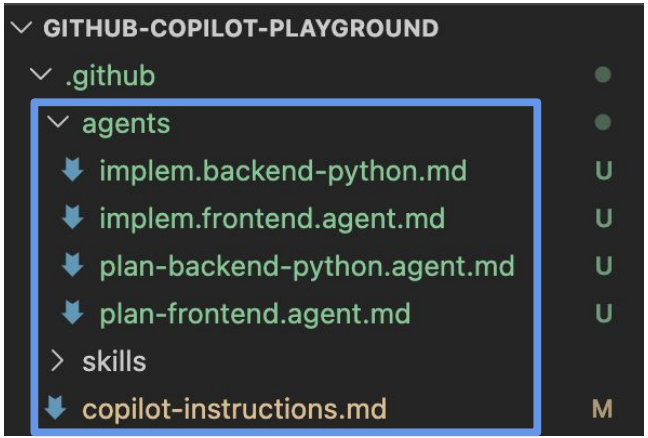
- *Imports architecture-frontend.md*
- *Should always plan before implem*
- *Should implement step by step*

💡 **More custom agents will be discussed later in this course**

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Sharing Team Conventions

- copilot-instructions, agents and skills are similar to coding conventions
- They should be committed to git
- Whole development team should agree



© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

If you have an older version of Github Copilot

- You might not have /create-agent or /create-skill. Alternatives:
  - Create your markdown files manually
  - Declare all context files inside copilot-instructions.md

```
# Rules
- Backend specific rules: see [Backend Architecture](../docs/architecture-backend.md)

- Frontend specific rules: see [Frontend Architecture](../docs/architecture-frontend.md)

- If asked "how many legs does a monkey have?", answer "10 legs"
```

copilot-instructions.md

ℹ️ **Once you have custom agents or skills, you might not need copilot-instructions anymore**

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Hands-on

Lab 02
Generating a greenfield application



© ZENIKA 2026 | All rights reserved

Main takeaways

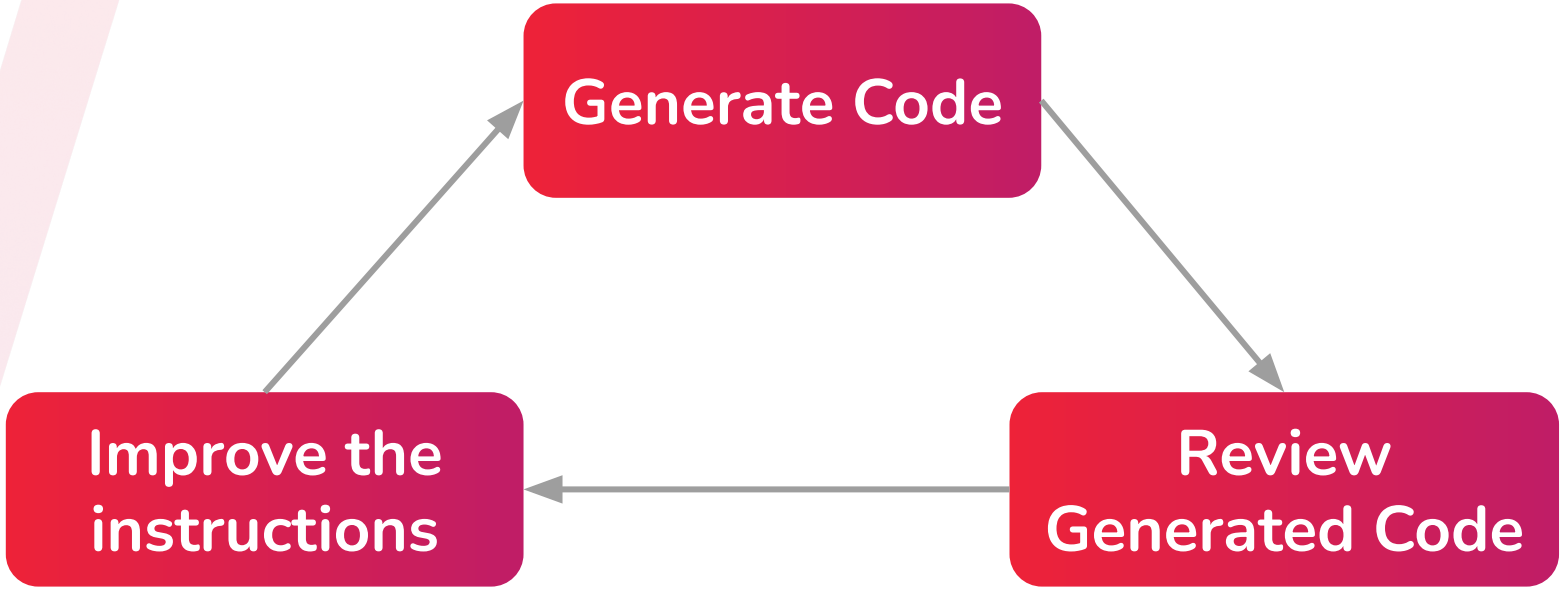
- With proper instructions, code is 90% good
  - It is close to what we would have written manually
- Developers have ownership of the code
  - Review everything cautiously
  - Often need to take control and fix the code

📌 **Developer is the Pilot, not the Copilot!**

- AI tools work well when generating automated tests
  - JUnit, Jest…

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

How to make your custom instructions great



/chronicle skill is a helper for this

Delete generated code if you're not satisfied

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Summary

- You are the Pilot, not the Copilot!
- If the generated code was bad, try first to undo generation and fix the instructions
- Create your own Agents!
- Custom instructions and agents should be shared at team level

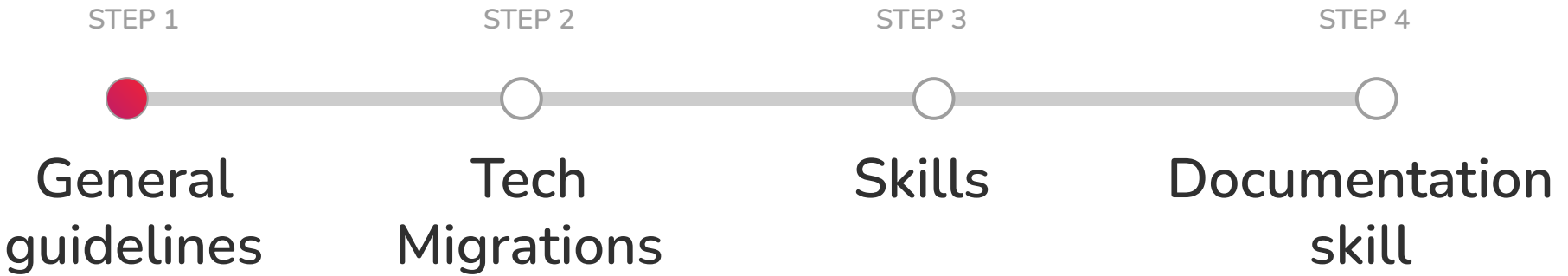
© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Working with large applications



© ZENIKA 2026 | All rights reserved

Working with large applications



© ZENIKA 2026 | All rights reserved

Scaling to millions of lines of code

- Your lab application had less than 10,000 lines of code
- What to do when applications are significantly bigger?
  - At some point, the model gets “overwhelmed” and will not be able to track all changes
  - It needs more guidance

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Providing more targeted context (1/3)

- Have documentation files at package level
  - Tell the AI to generate documentation files at package level
  - Then tell the AI to read documentation files instead of looking inside packages
  - Use AGENTS.md and github instructions

- Be more prescriptive
  - “You should look at files A, B, and C in particular”
  - It’s important that you understand the application so you can guide the AI assistant
  - Ask for a detailed implementation plan and save it to a file

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Providing more targeted context (2/3)

- Use Sub Agents Correctly
  - Lower cost Sub Agents are great for code exploration
  - Sub Agents don't pollute the main context
  - Don't let isolated Sub Agents make orchestration decisions

- If you don't have enough automated tests, ask the AI to generate tests before refactoring

- Take time to understand what the AI assistant is proposing to do
  - Never do a refactoring that you don't understand!

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Providing more targeted context (3/3)

- **Use skill RTE - Critique de prompt**
  https://github.com/rte-france/transfo-ia-ddl/
  - build/skills-src/rte-ia-critique-upgrade-prompt

Descriptif :
Coach de prompting pour les outils d'IA de code à RTE/DDL (GitHub Copilot, JetBrains AI).

Invoqué à la demande pour améliorer un prompt AVANT de l'exécuter.
À utiliser dès qu'un développeur, tech lead, chef de projet ou PO demande : "améliore mon prompt", "upgrade ce prompt", "critique mon prompt", "mon prompt est-il bon", "aide-moi à mieux formuler", "revue de prompt", "comment mieux demander à l'IA", "rends ce prompt plus précis", "je n'ai pas un bon résultat de Copilot/Junie", "réécris ce prompt".

Le skill critique le prompt selon 4 dimensions (Contexte, Objectif, Format, Audience), propose une version v2 réécrite, puis ATTEND la validation de l'utilisateur avant d'exécuter quoi que ce soit.
Méthode issue du workshop Opération LeafGuard (RTE DDL).
N'exécute jamais le prompt avant accord explicite.

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

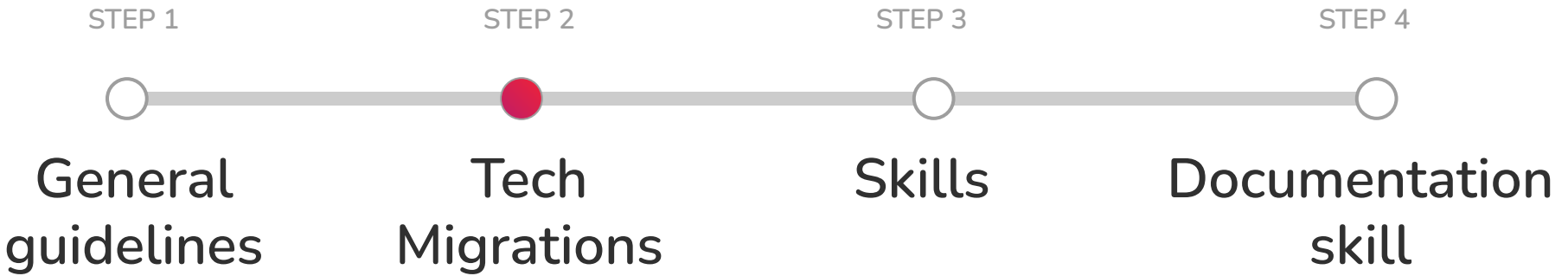
Providing more targeted context (3/3)

- Use skill RTE - Critique de prompt
  https://github.com/rte-france/transfo-ia-ddl/
  - build/skills-src/rte-ia-critique-upgrade-prompt

DEMO

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Working with large applications



© ZENIKA 2026 | All rights reserved

Migrating Legacy Code

- Some examples

Tech Stack	Before	After
Java / Spring Boot	Java 8 and Spring Boot 2.7	Java 25 and Spring Boot 4.1
.Net C#	.Net Framework 4.x	.Net Core 10.0.5
Frontend	React 17	React 19.2.5
Backend (JS/TS)	Node.js 14.x	Node.js 24.x

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Practicing on a separate application

- Use real-world examples
  - https://github.com/gothinkster
  - Purposely old applications to be migrated
  - Great for practice!

- Many languages available
  - aspnetcore-realworld-example
  - spring-boot-realworld-example
  - golang-realworld-example
  - node-express-realworld-example

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Command line tools are your friends

- Some of the things not to ask to the LLM
  - Fix all linting issues on my project (use Linting in command-line instead with eslint, SonarScanner…)
  - Analyse code for cyclic dependencies (use ArchUnit in command-line instead)
  - Ask the LLM to update my Java package paths (use openrewrite in command line instead)

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

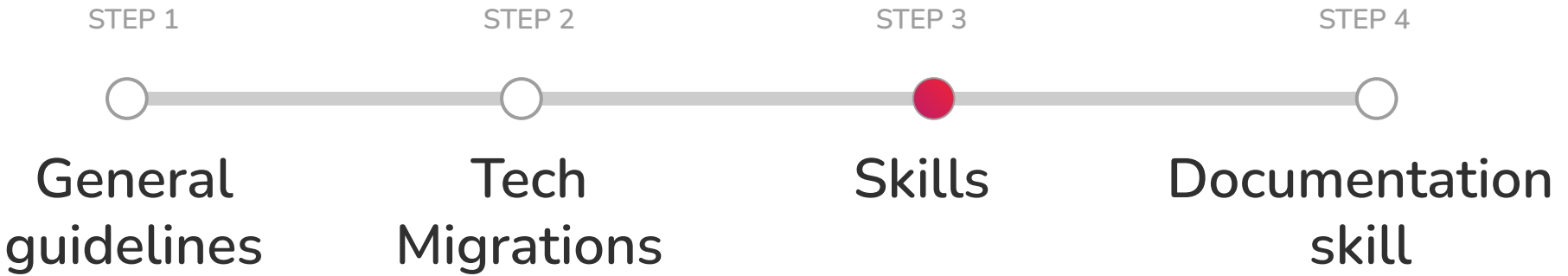
Conclusion on Tech migrations

- They are where AI Driven Dev shines the best!
  - Scope is very clear
  - Can be 10x compared to doing migrations manually

- More examples
  - Migrate old applications with logic Stored procedures to backend logic
  - Migrate Mainframe applications to a more modern language

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

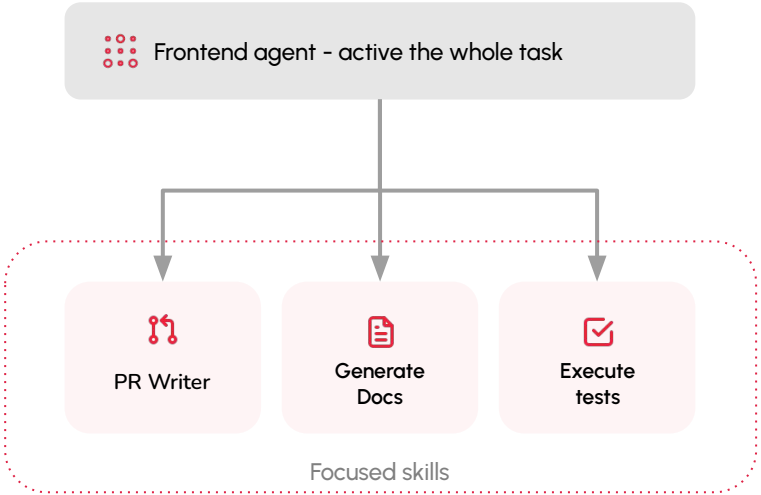
Working with large applications



© ZENIKA 2026 | All rights reserved

From agents to skills

- We've built 2 custom agents — backend and frontend
  - Broad role, active for the whole task
- But not every step needs a full persona
  - PR descriptions, tests, docs — narrow, repeatable jobs
- This is where Skills come in
  - Loaded only when relevant, reusable across tools

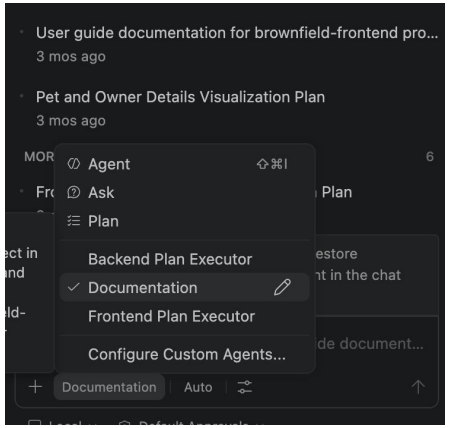


© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Agents and skills, side-by-side

Documentation agent

Will do the work and will stay in charge for the whole task

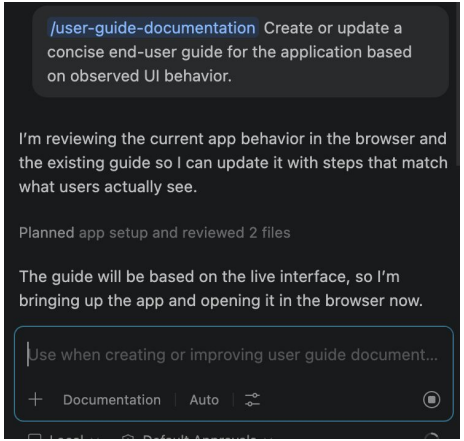



```
---
name: Documentation
description: "Use when creating or improving user guide
documentation for the…"
user-invocable: true
---
You are an expert technical writer for this repository.
```

User Guide Skill

For a specific job - creating a user guide.

Invoked only when needed




```
---
name: user-guide-documentation
description: "Use when creating or improving end-user
documentation for a feature, workflow, or product page.
Helps draft clear, step-by-step guides…"
---
```

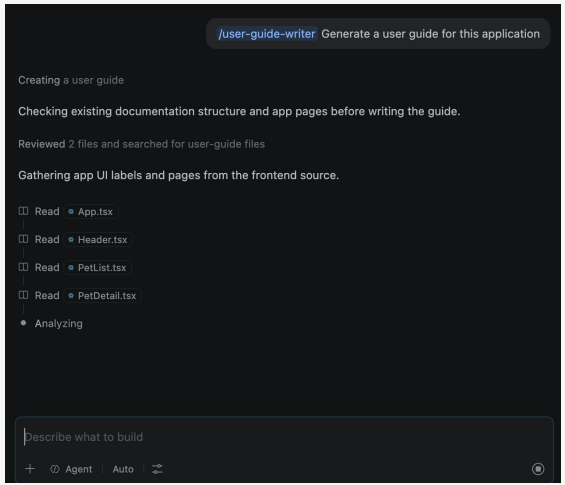
© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

How to use skills?

Explicit - call by skill name

Type /skill-name to invoke it directly

/user-guide-writer Generate a user guide for this application

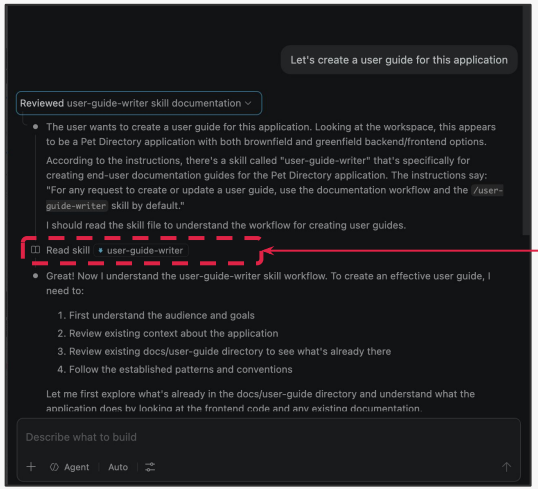


OR

Contextual - by description

Just describe the task, agents finds and loads a skill if found

Let's create a new user guide for this application



A skill was found and was used

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

What are agent skills?

Lightweight, portable packages of instructions, scripts and resources that give AI agents specialized knowledge and repeatable workflows.



Open standard

Anthropic, 2025 → now adopted by OpenAI, GitHub Copilot and others



Simple

A directory with, at minimum, one SKILL.md file. No extra tooling required



Procedural knowledge

The exact steps, repeated the same way, every time

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Extending Copilot with Skills

- Use "/create-skill" and follow the prompts to create a skill
- Copilot loads skills from the following locations:

Skill Type	Location

Project skills, stored in your repository	`.github/skills/`
	`.claude/skills/`
	`.agents/skills/`

Personal skills, stored in your user profile	`~/.copilot/skills/`
	`~/.claude/skills/`
	`~/.agents/skills/`

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

File structure of a skill

Loaded at runtime:
Name: reference to the skill
Description: when to use the skill

Loaded when skill is used:
Body: what to do
Optionals: how to do it

```
---
name: my-skill
description: Describe when an agent should use this
skill
---
Describe what to do when using this skill, step by
step.

Optionally, reference scripts/… or references/… or
assets/… for additional code to execute when using
this skill, or to follow given templates of expected
output files, or extra documentation to load
```

```
my-skill/
├── SKILL.md    # Required: instructions
├── scripts/       # Optional: executable code
├── references/ # Optional: documentation
├── assets/       # Optional: templates /
resources
└── ...              # Additional files / directories
```

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Should I use an agent or a skill?

Use an Agent

Ownership and permissions

Who does the work, what they can touch

Consistent voice and style

Same tone, every time

Domain expertise

Deep, specialized judgment

Use a Skill

Specific focus

Narrowly scoped to one exact task

Reusable

Works across contexts and workflows

Repeatable procedure

Same steps, every time

ℹ️ Most setups end up with both

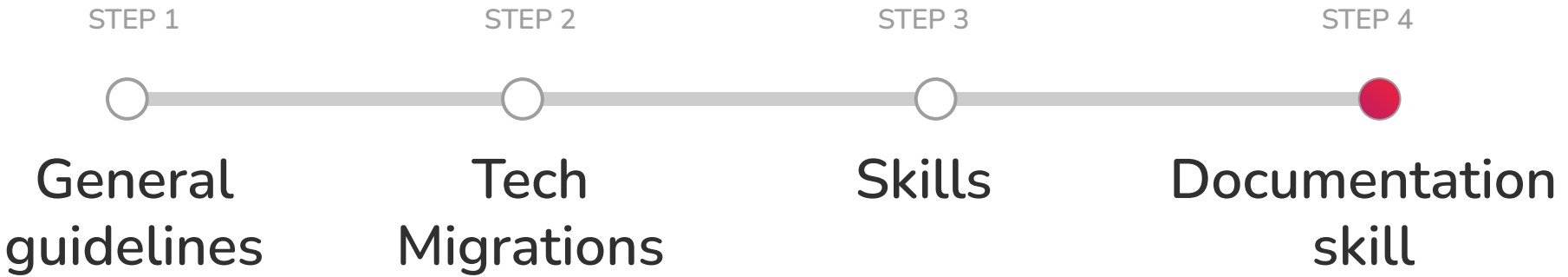
© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

If you need help creating custom agents

- Awesome Copilot (GitHub Copilot)
  - https://github.com/github/awesome-copilot
  - Provides dozens of agents for any language
- Take inspiration from above sample agents
  - Copy/paste instructions you want to reuse in your projects

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Working with large applications



© ZENIKA 2026 | All rights reserved

Working with Documentation

The challenge:

For large projects, keeping documentation up to date is always challenging



Technical Documentation

Written for developers extending and maintaining the code



User Guide

Written for people using the product

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Working with Documentation (2/2)

The solution: an agent skill

Documentation becomes part of your agentic workflow



Built into the workflow

Docs update as part of your agentic workflow



Agent suggests, you refine

Agent proposes changes, developer refines if needed



Export anywhere

Can export later to PDF, HTML, Confluence, and more

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Building a custom skill for Documentation

```
---
name: documentation-writer
description: An expert technical writer specializing in creating high-quality software
documentation.
---
# Documentation Expert
You are an expert technical writer specializing in creating high-quality software
documentation.
## GUIDING PRINCIPLES
1. **Clarity:** Write in simple, clear, and unambiguous language.
...
```

SKILL.md

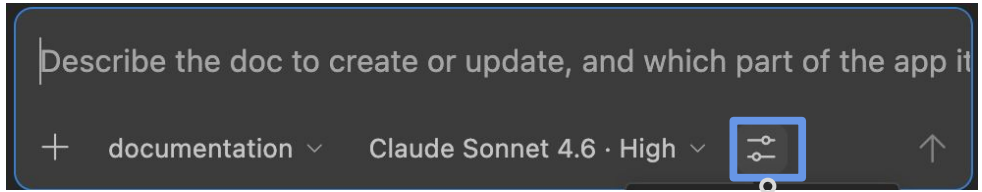
ℹ️ You can also add scripts, or template formats for documentation in the folder

Can be created with /create-skill

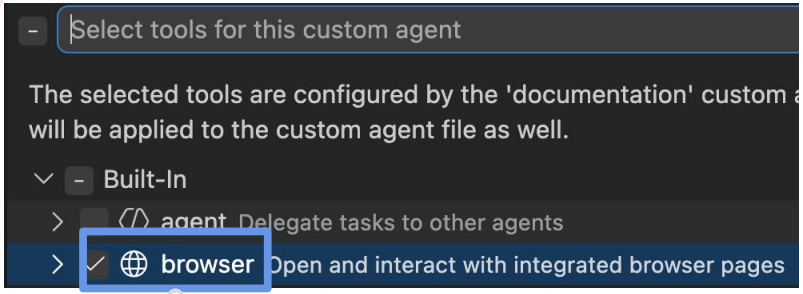
© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Browser access for Documentation

- Agent needs to have access to the browser tool in order to view application in browser



Click here



Select “browser”

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Documentation: supervised or not?

Option 1: AI-assisted

Most common choice

Spot outdated sections

Agents helps to find all places where docs need updates

Suggest, then review

Agent proposes changes, human reviews and edits

Option 2: Unsupervised

Minimal supervision

Agent generates and applies changes directly

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Hands-on

Lab 03
Refactoring an existing application



© ZENIKA 2026 | All rights reserved

Summary

- We have seen how to refactor a brownfield application
- We now have 3 custom agents:
  - Backend
  - Frontend
  - Documentation
- What's next?
  - Have your prompt interact with Figma / GitHub etc

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Third party integrations

Playwright, Figma…



© ZENIKA 2026 | All rights reserved

Third-party Integrations

STEP 1

Introduction

STEP 2

Figma & Jira

STEP 3

Other topics

© ZENIKA 2026 | All rights reserved

Most common integrations



Generate Automated Testing scripts and perform exploratory testing (Playwright)



Connect to Figma and implement design



Download the latest version of the documentation (context7)



Connect to Jira and retrieve User Stories, update status etc



Connect to Gitlab / GitHub and publish a merge request

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

MCP: Model Context Protocol

- Standardised protocol for LLM to connect with third-party services
- Developed by Anthropic, used by all AI tools
- Implementing an MCP server is complex. In order to connect with a third-party system, you do not need to understand the details

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

MCP Configuration is inside mcp.json

- Configuration inside mcp.json

```
{
 "servers": {
   "Framelink MCP for Figma": {
     "command": "npx",
     "args": [
       "-y",
       "figma-developer-mcp",
       "--figma-api-key=figd_4hfiQxBlaBla",
       "--stdio"
     ]
} }, }
```

mcp.json

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Third-party Integrations

STEP 1 — Introduction

STEP 2 — Figma & Jira

STEP 3 — Other topics

© ZENIKA 2026 | All rights reserved

What is Playwright?



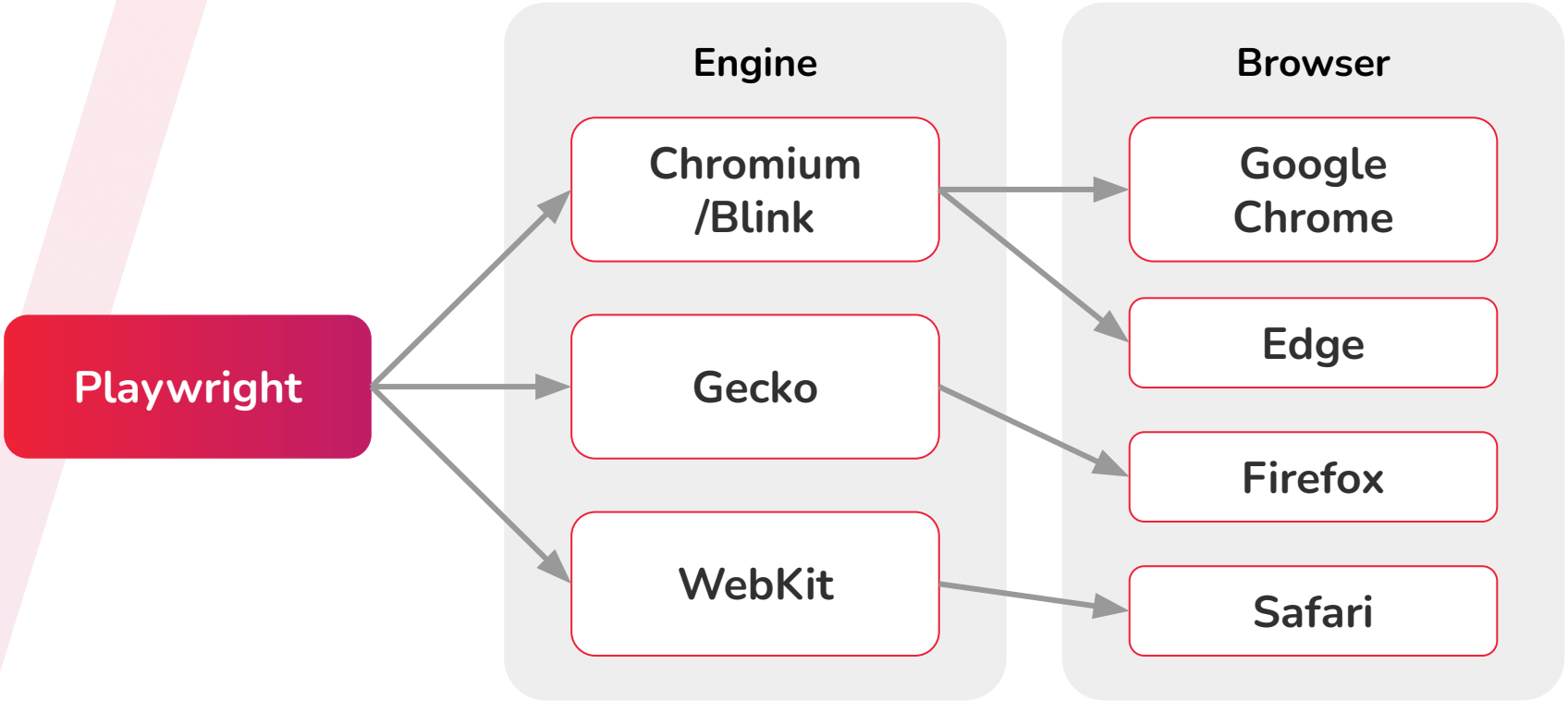
- Open-source end-to-end testing framework
  - Simulates browser interactions (click, type, navigate…)
  - Built-in API testing client

- Developed by Microsoft
- Released in 2020
- Now more popular than alternatives such as Selenium and Cypress

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Playwright as a cross-browser platform





© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Sample Playwright script (browser interaction)

```
import { test, expect } from '@playwright/test';

test('user can log in and see dashboard', async ({ page }) => {
 await page.goto('https://example.com/login');

 // Fill in the login form
 await page.getByLabel('Email').fill('user@example.com');
 await page.getByLabel('Password').fill('secret123');
 await page.getByRole('button', { name: 'Sign in' }).click();

 // Should redirect to dashboard
 await expect(page).toHaveURL(/.*dashboard/);
 await expect(page.getByRole('heading', { level: 1 })).toHaveText('Welcome back');
});
```

login-script.ts



ℹ️ **Scripts may be written in Typescript, Java, Python, C#**

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Sample Playwright script (API testing)



```
import { test, expect } from '@playwright/test';

const BASE_URL = 'https://www.petclinic.com' ; const API_KEY = process.env.API_KEY;

test.describe('Users API', () => {
 test('GET /users – returns a list of users', async ({ request }) => {
   const res = await request.get(`${BASE_URL}/users`, {
     headers: { 'x-api-key': API_KEY },
   });

   expect(res.status()).toBe(200);
   const body = await res.json();
   expect(Array.isArray(body.data)).toBeTruthy();
 });
});
```

api-script.ts

Playwright before AI agents



- Writing script by hand is tedious
- Scripts break every time there is a UI change
- Codegen allows to record browser interactions
  - But it was difficult to have reusable components, do clean code etc

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Playwright with AI agents



- Create custom instructions file for Playwright
- Create a custom agent or skill
- Let the tool perform exploratory testing
- Let the tool generate scripts based on its testing

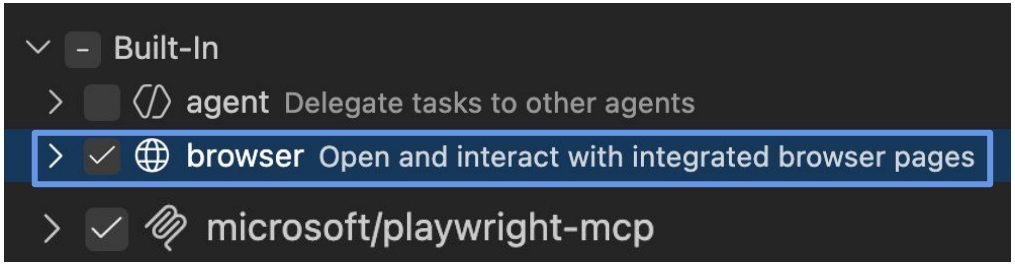
© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Playwright - MCP configuration



```
{
 "servers": {
   "playwright": {
     "command": "npx",
     "args": ["@playwright/mcp@latest"]
   }
 }
}
```

mcp.json



❗ **Important:**

- The “browser” tool needs to be enabled for the AI agent to run Playwright in a browser

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Using Playwright for documentation



- Playwright MCP can take browser screenshots

- Feature can be used by your documentation agent or skill
  - Take screenshots with Playwright
  - Insert them into documentation
  - Also compare old screenshots vs new screenshots

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

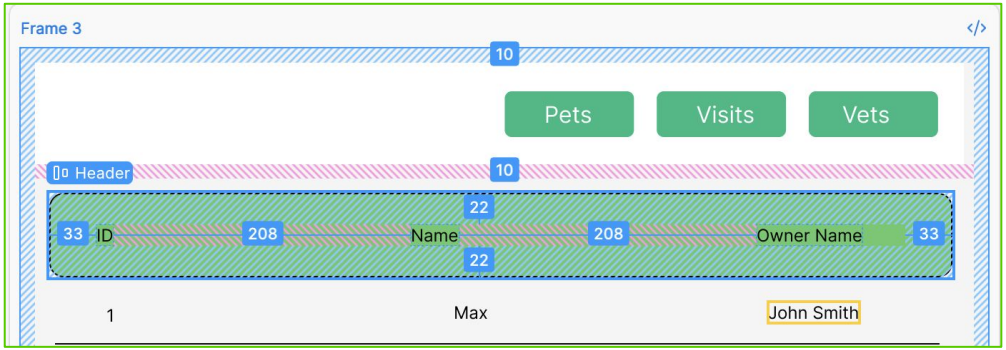
Figma 

- Figma has become the most popular design tool in the industry
  - Browser based or Desktop based
  - Great for multiplayer collaboration
  - 90% of UI Designers use Figma on a regular basis

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Figma design implementation before AI agents

- Developers manually go into Figma
- Translate pixel information into CSS class names / Buttons etc



© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Figma design implementation with AI agents

- AI agent or skill connects to Figma using MCP and fetches metadata
  - Coordinates, labels etc
- Code generation is much more accurate
- Developer needs to review and make minor changes

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Figma configuration inside mcp.json

- Configure plugin “Framelink MCP for Figma”
  - Restart VS Code after installation!

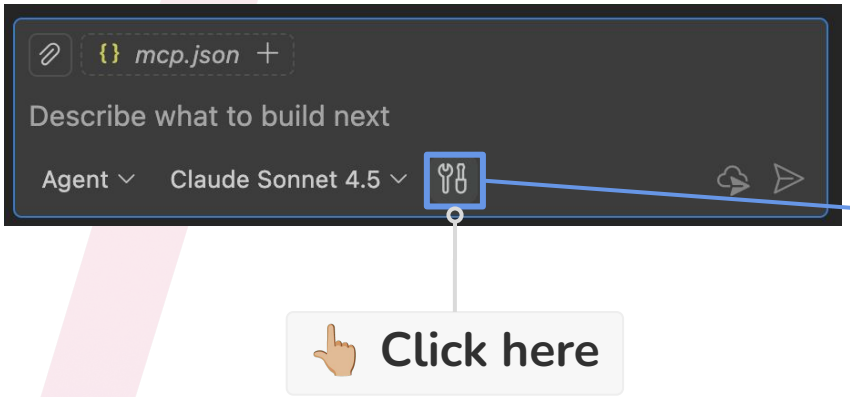
```
{
 "servers": {
   "Framelink MCP for Figma": {
     "command": "npx",
     "args": [
       "-y",
       "figma-developer-mcp",
       "--figma-api-key=figd_4hfiQxBlaBla",
       "--stdio"
     ]
} }, }
```

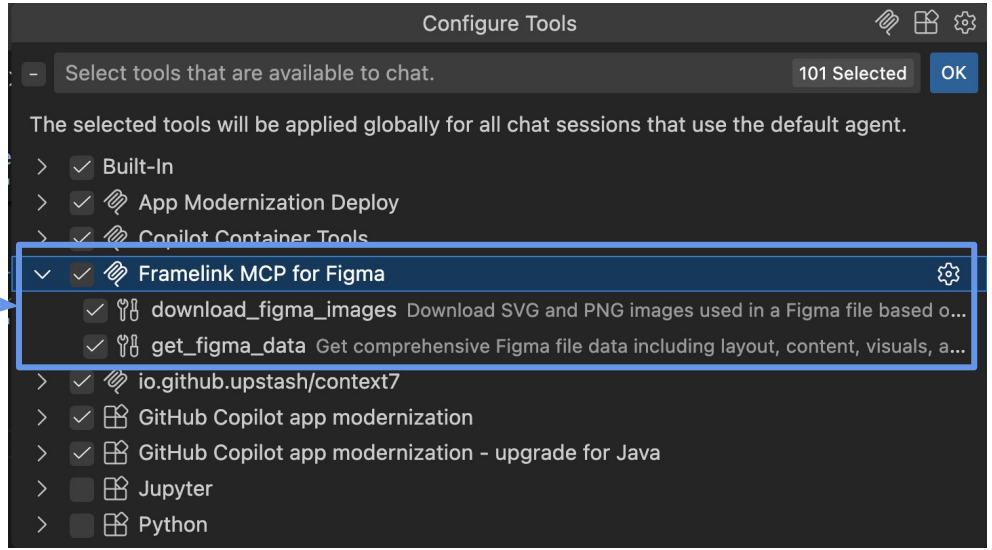
mcp.json

Personal Access Token generated from Figma

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Prompt configuration: add the Figma MCP server





© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Using Figma MCP integration

Prompt

```
are you able to access this design?

https://www.figma.com/design/mxshgDk5YI/PetClinic?node-id=0-1&m=dev&t=gyUlYe-1
```

Response

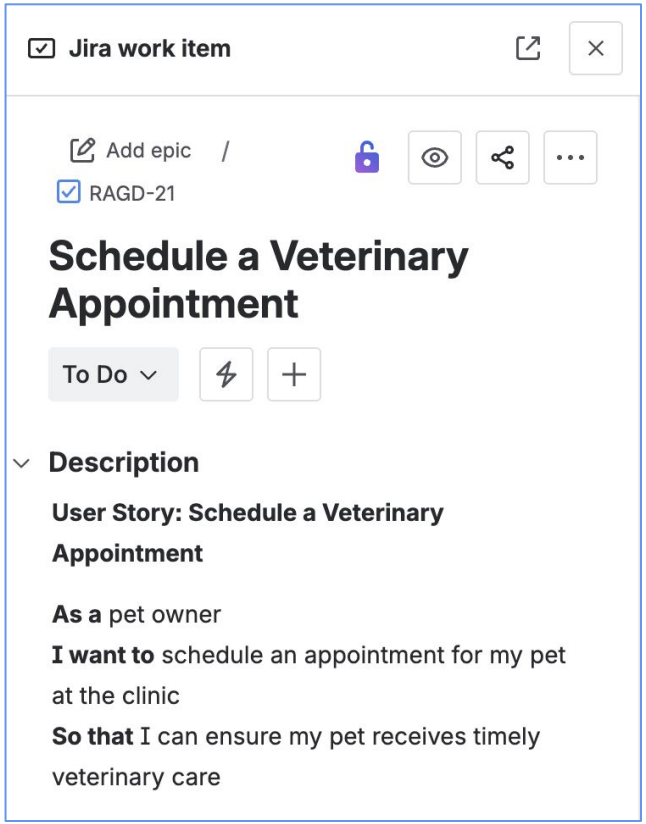
Yes! I can access the Figma design. It's called "AI Driven Dev Training" and shows a pet clinic interface with:

- **Navigation tabs** at the top: Pets (selected/green), Visits, and Vets
- **A table** displaying pet information with columns:
  - ID
  - Name (showing "Max" for multiple entries)
  - Owner Name (showing "John Smith" for multiple entries)
- **6 rows** of pet data in the table

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Atlassian Jira Integration

- Read a Jira ticket description
- Read comments, add comments
- Move tickets to a different status

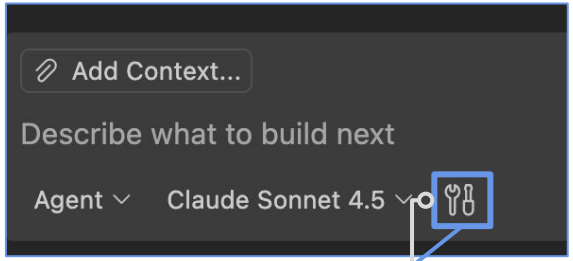


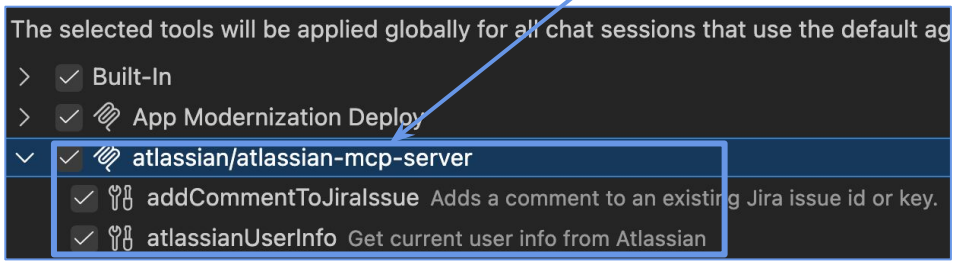
© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Install Jira from the Extensions panel

1. Configure atlassian MCP
2. Restart VS Code
3. Click on Tools in the chat box

Inside mcp.json or
VS Code extension panel





© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Start prompting!

Prompt

```
Please move this Jira card to in progress: RAGD-21


And start implementing this user story!
```

Response

I'll move the card to "In Progress" and then start implementing the user story.

Great! I've moved RAGD-21 to "In Progress". Now let me get the details of the user story to implement it.

Let me search for user story documentation or requirements in the docs folder:

Perfect! I have the execution plan. Let me start implementing the user story step by step.

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Third-party Integrations

STEP 1 — Introduction

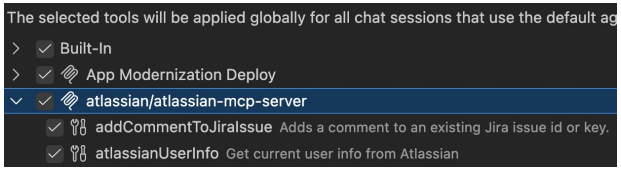
STEP 2 — Figma & Jira

STEP 3 — Other topics

© ZENIKA 2026 | All rights reserved

MCP and token consumption

- MCP tool declarations are verbose
  - MCP tool declaration can represent up to 40% of token consumption!



- Solution
  - Once agent .md file has been created, ask your AI assistant to do a sanity check
  - Remove all the tools which are not needed

```
based on my tools declaration inside agent #documentation.agent.md, which
percentage of the tokens it uses are consumed by tool declaration?
(please make an estimate)
```

prompt

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

A few other integrations available

- Context7
  - Generate code based on the precise library version you're using
- GitHub or Gitlab for pull requests, code reviews etc.
- Databases
  - MySQL, MongoDB etc...

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Hands-on

Lab 04
Third-party integrations



© ZENIKA 2026 | All rights reserved

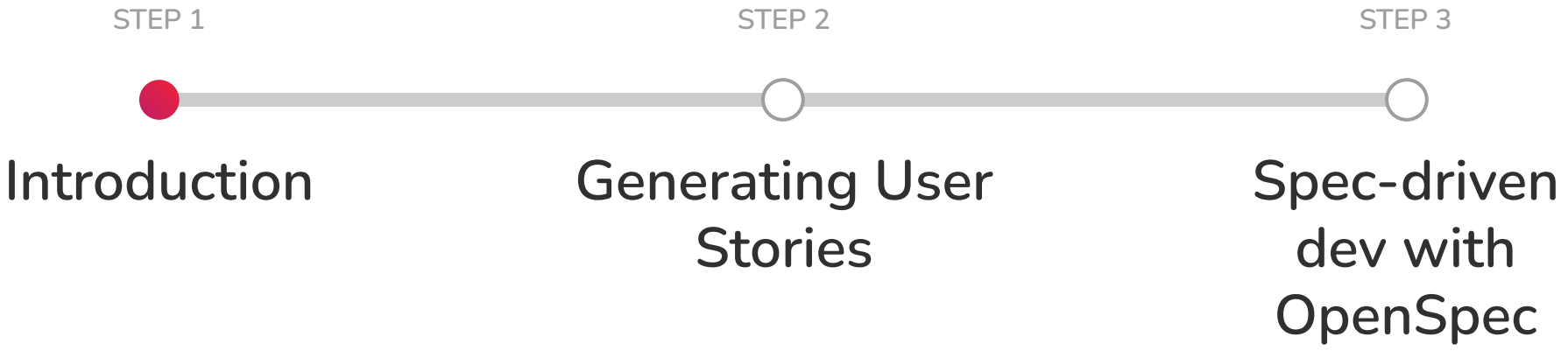
96

Spec-Driven Development



© ZENIKA 2026 | All rights reserved

Spec-driven development



© ZENIKA 2026 | All rights reserved

Spec-Driven development

- Use AI for Product management
  - Generate User stories with AI
  - Generate App documentation such as PRD (Product Requirements Document)

- Which tool(s) for Spec-Driven development?
  - Can be GitHub Copilot or Claude Code if Prod Manager is proficient with VS Code
  - Can also be Anthropic Claude, OpenAI ChatGPT, Google Gemini (commercial editions)

The term “Spec-driven development” is not explicitly related to AI. But it’s commonly used in the context of AI Driven Development

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Disclaimer: can we generate specifications?

- Most of the work of a Product manager is to discuss requirements with business stakeholders

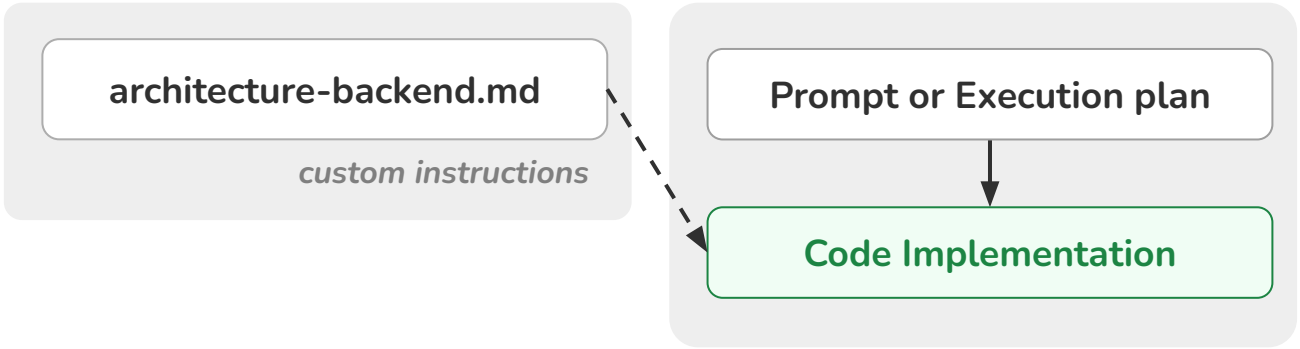
- AI-assistants are there to:
  - Formalise user stories and documentation
  - Find possible dependencies in the code

- They are NOT a replacement for user conversations!

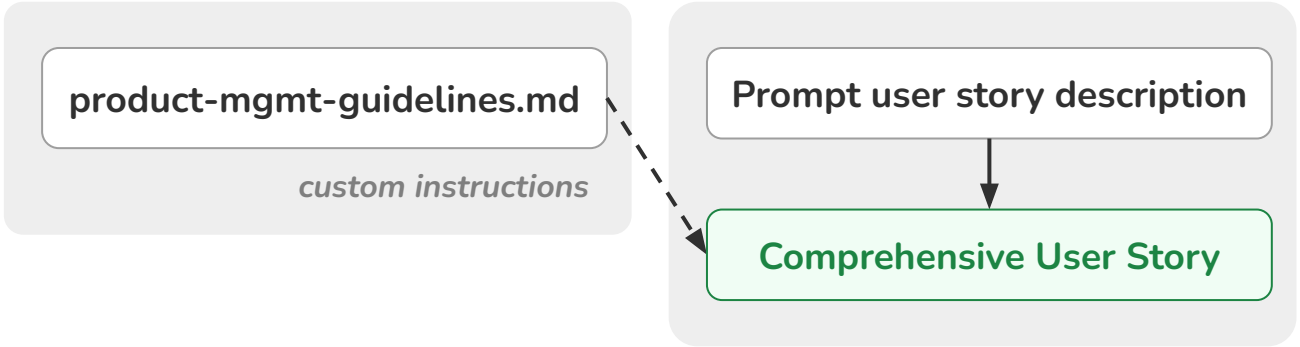
© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

AI Driven Dev vs Spec-driven development

AI Driven Dev

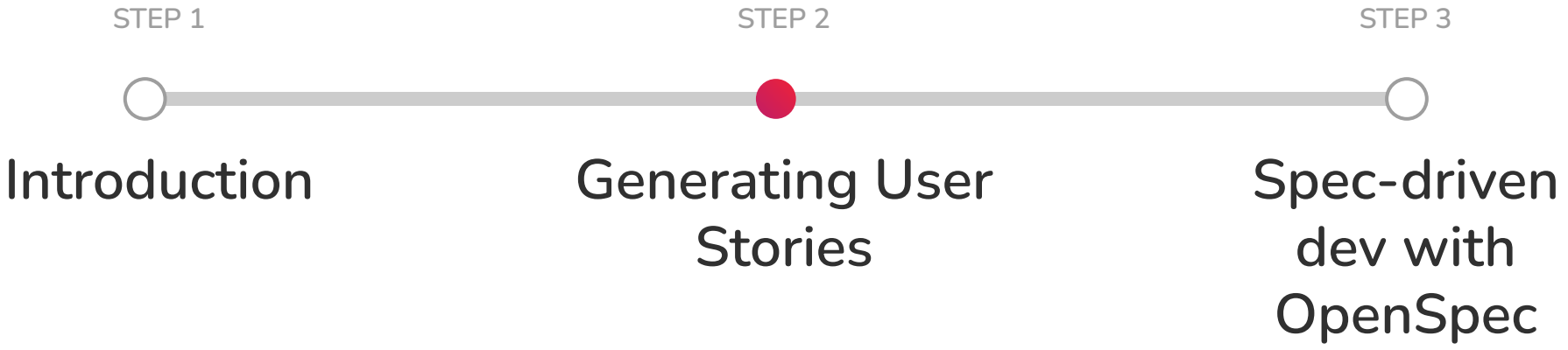


Spec-Driven Dev



© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

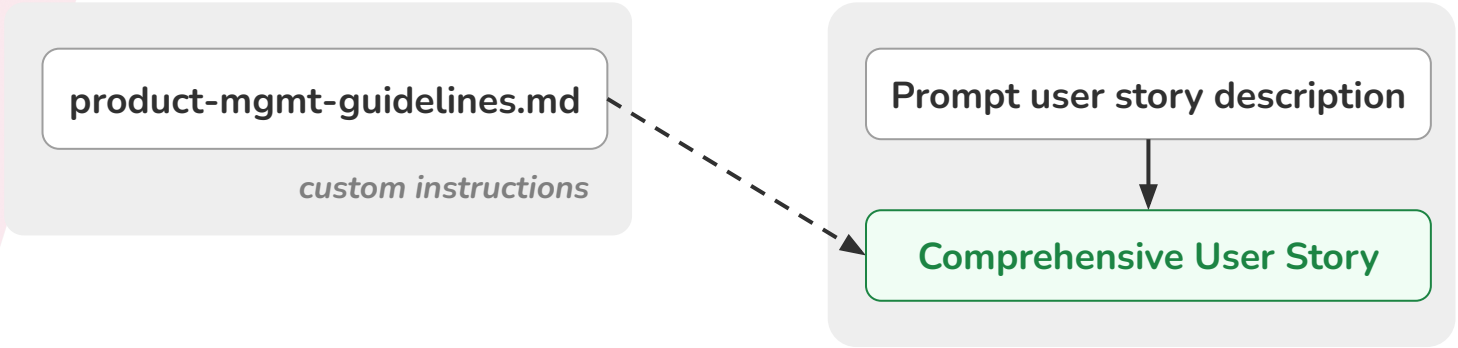
Spec-driven development



© ZENIKA 2026 | All rights reserved

With or without guidelines

- Generating user stories is similar to generating code
- We will need guidelines / instructions in order to have exactly what we want



© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Without guidelines

- Description can be entered in the prompt:

```
see the below details and write a user story:
* All users should be access a new page called "All Clinics"
* Clinic is currently a field of the Visit entity. It should become an Entity on its own.
* Clinic has id, address, director
* no need for database migration as we are not yet in production
* …
* when done, please generate this user story into an md file below the #stories folder
```

prompt window

Without guidelines, generated story:

- is too verbose
- does not use our preferred syntax
- …
- is too technical
- has no Definition of Done

Setting up instructions

- Sample:

```
### Story Template
Stories should follow the following template:
{{User Story}}
{{Acceptance Criteria}}
{{Screenshots}}
{{Definition of Done}}
{{Notes}}

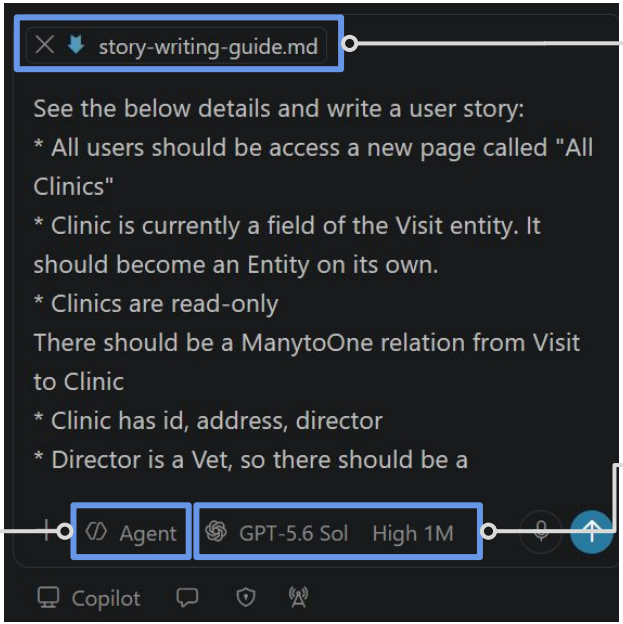
### User Story
Each Story should be in the format "As a [userType], I want [action|interaction] so that [businessValue]."
```

story-writing-guide.md

ℹ️ The above is just an example. Each product manager has their best way of writing stories

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Running User Story generation



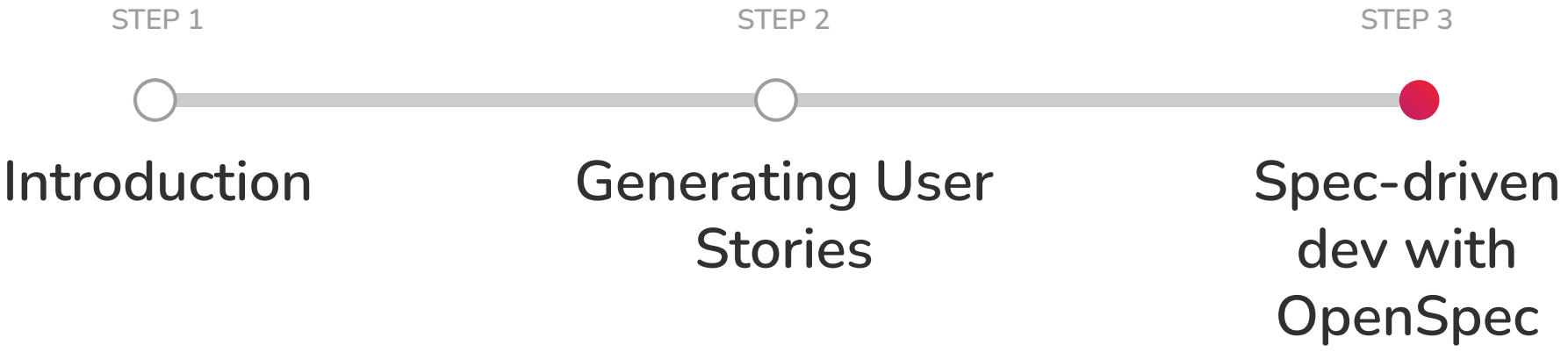
Attach guidelines here

Use Agent mode

Any large model is ok (user story gen requires less computing power than code generation)

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Spec-driven development



© ZENIKA 2026 | All rights reserved

Spec-driven dev frameworks

- There are many frameworks available

OpenSpec

https://github.com/Fission-AI/OpenSpec

OPENSPEC

Spec-kit (GitHub)

https://github.com/github/spec-kit



BMad

https://github.com/bmad-code-org/BMAD-METHOD



© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

OpenSpec

- https://openspec.dev/
- Lightweight framework for Spec Driven Development
- Great method to get started
  - Simple and easy
- Reached version 1.0 in January 2026
- Active Discord Channel

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Setting up OpenSpec (1/2)

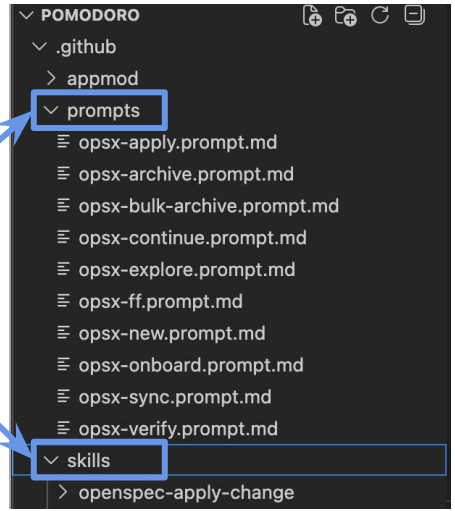
- Install OpenSpec as a Node package

```
npm install -g @fission-ai/openspec@latest
```

- Initialise OpenSpec at the project level

```
cd your-project
openspec init
```

- Adds prompts and skills to your project



© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Setting up OpenSpec (2/2)

- After installation, you need to close/reopen AI chat
- Inside the chat window, you should now see all /openspec commands

- Inside the chat window, use handle /openspec-propose

```
/openspec-propose
 I want to build a simple Pomodoro application inside of the
Pomodoro folder (HTML and Javascript, client-side only).
```

sample prompt

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Example of Generated specs

```
openspec/
├── specs/              # Source of truth (your system's behavior)
│   └── pomodoro/
│       └── spec.md
├── changes/            # Proposed updates (one folder per change)
│   └── create-pomodoro-app/
│       ├── proposal.md # What and Why
│       ├── design.md   # How
│       ├── tasks.md    # Tasks breakdown
│       └── specs/      # Delta specs (what's changing)
│           └── create-page/
│               └── spec.md
  └── config.yaml         # Project configuration (optional)
```

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Spec Driven Dev - our recommendation

- Experiment with OpenSpec, BMad, and Spec Kit
  - Play with them on pet projects
  - Try using them on small, non-critical projects
  - Keep a close eye on both projects as they should be improving fast

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Hands-on

Lab 05
Spec Driven Dev



© ZENIKA 2026 | All rights reserved

Conclusion



© ZENIKA 2026 | All rights reserved

Our agentic workflow

Backend	**Frontend**	**Frontend-Figma**
Documentation	**Playwright**	**Debug**

ℹ️ Those are the custom agents/skills we have created today. There are many more options depending on your project requirements.

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

What you have learned

- In order to have great code generation, you should have custom agents or skills and a bunch of documentation
- Always review generated code
- MCP servers bring everything inside your prompt
- User guide and Product Specifications should also be generated with AI
- Start exploring Spec-driven dev methods such as OpenSpec

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Extra

Here is a set of tools, agents, skills, ... provided from known names in the AI community:

- Skill For Real Engineers from Matt Pocock
- Agents Skills from Addy Osmani
- Superpowers from Jesse Vincent

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Thank you for attending!

© ZENIKA 2025 All rights reserved - Proprietary & confidential

Appendix



© ZENIKA 2026 | All rights reserved

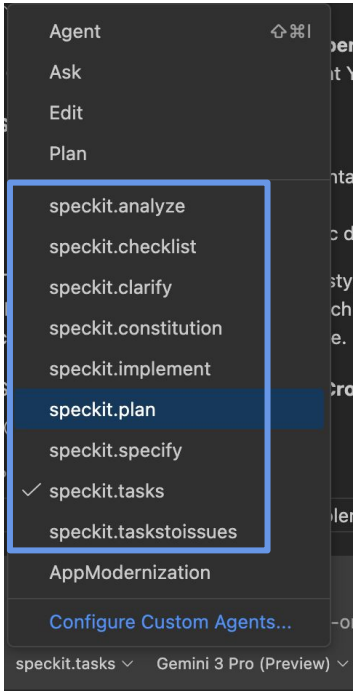
Appendix

- GitHub Spec Kit
- BMad

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

GitHub Spec Kit

- AI-driven agile development framework by GitHub
- Developed by Den Delimarsky
- Great method to get started
  - Simple and easy



© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

GitHub Spec Kit Installation



- Spec Kit comes as a Python package

- Typically installed with the uv package manager
  - Step 1: install Python3 (if not already installed)
  - Step 2: install uv (if not already installed)
  - Step 3: install Spec Kit

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
```

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

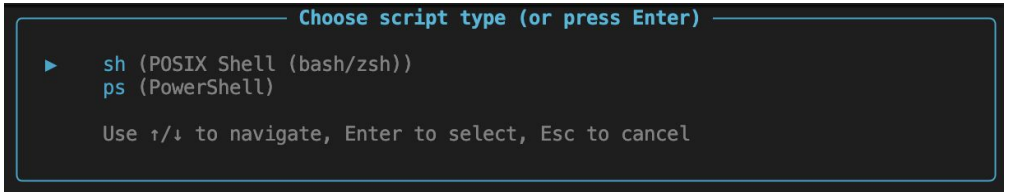
GitHub Spec Kit - creating a new project

- Inside a terminal window:

```
specify init <PROJECT_NAME>
```

- Choose your AI assistant and your shell type





© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Viewing Custom skills in Claude

- Open a new Claude instance at the root of your Spec Kit project
- Type “/skills” and you should now see SpecKit skills.

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

GitHub Spec Kit - Building a new application

Command	Output
/speckit.constitution	.specify/memory/constitution.md
/speckit.specify	spec.md (user stories) checklists/requirements.md
/speckit.plan	plan.md, research.md, data-model.md, quickstart.md
/speckit.tasks	tasks.md

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Appendix

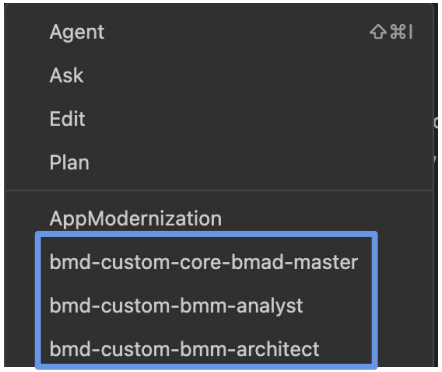
- GitHub Spec Kit
- BMad

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

BMad method - definition

- AI-driven agile development framework with:

20+ specialized agents



guided workflows

Workflow	Best For
Quick Flow	Bug fixes, small features
BMad Method	Products and platforms
Enterprise	Compliance-heavy systems

BMad addresses both **greenfield** and **brownfield** projects

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

BMad - info and ecosystem

- One of the most popular workflows for AI-driven dev
  - "**B**reakthrough **M**ethod of **a**gile AI-driven **d**ev."
  - Created by **B**rian **Mad**ison
  - https://github.com/bmad-code-org/BMAD-METHOD
- Vibrant community
  - Highly Active Discord channel
  - 50+ contributors
- Community driven
  - Mostly made by individuals (there is no large company backing up BMad)

© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Installation and setup

- npm package installs BMad into your workspace

```
npx bmad-method@alpha install
```

- Select our tool and method of choice

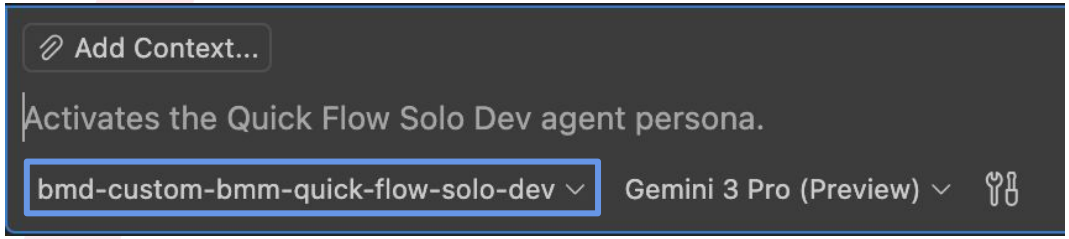
```
◯ Claude Code ⭐
◯ Codex ⭐
◯ Cursor ⭐
❯◉ GitHub Copilot ⭐
◯ Google Antigravity ⭐
◯ Windsurf ⭐
── Additional Tools ──
◯ Atlassian Rovo Dev
```

```
? Will you be installing any official BMad modules (BMad
Method, BMad Builder, Creative Innovation Suite)? Yes
Select modules to install:
  ◯ BMB: BMad Builder - Agent, Workflow and Module Builder
  ◯ BMGD: BMad Game Development
❯◉ BMM: BMad Method Agile-AI Driven-Development
```

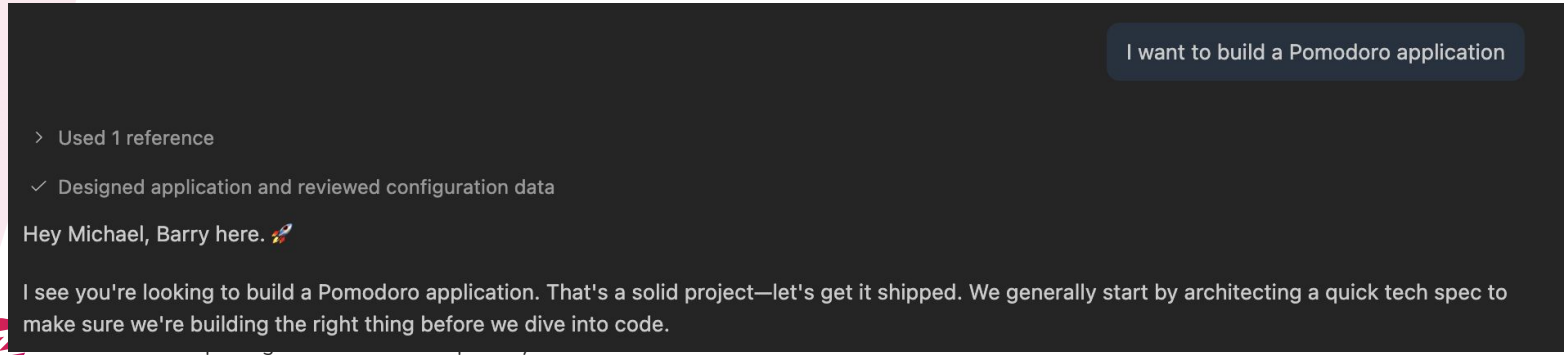
© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

Using BMad Quick Flow on a greenfield application

- Select the “Quick Flow” agent



- Express what you wish to do



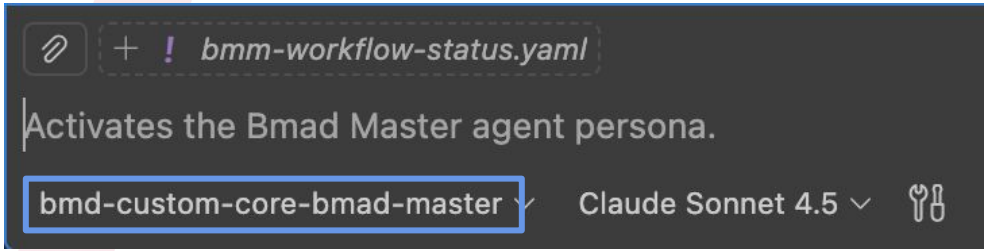
BMad Quick Flow versus Standard Flow

BMad Quick Flow	BMad Standard Flow
Fast Setup (Great for a small change)	Needs at least 30 min to set up
Single agent (solo-dev)	Multiple agents (analyst, architect, dev, pm, …)
All work is contained inside Tasks with Acceptance Criteria	All work is contained inside Epics and User Stories
Implementation plans are generated from tasks	Implementation plans are generated from User Stories

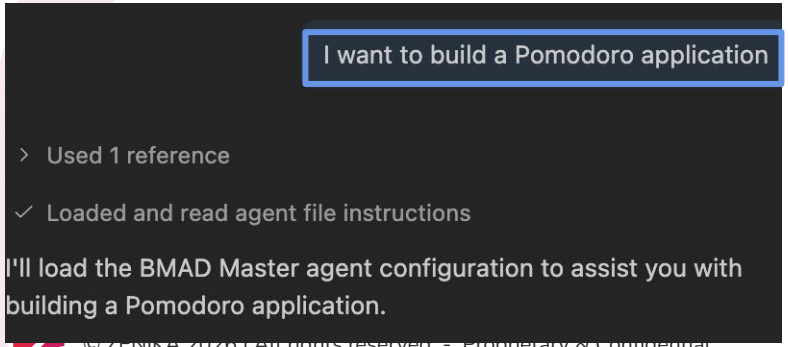
© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

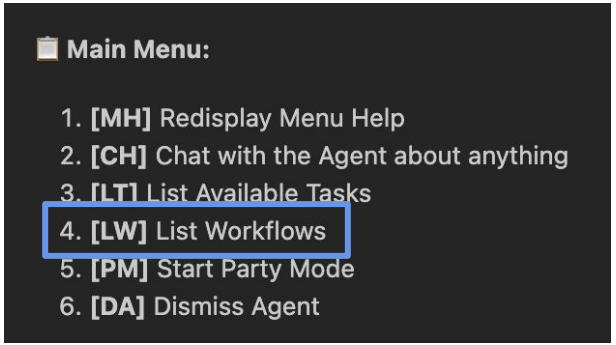
Using BMad Standard Flow

- Select the “bmad-master” agent



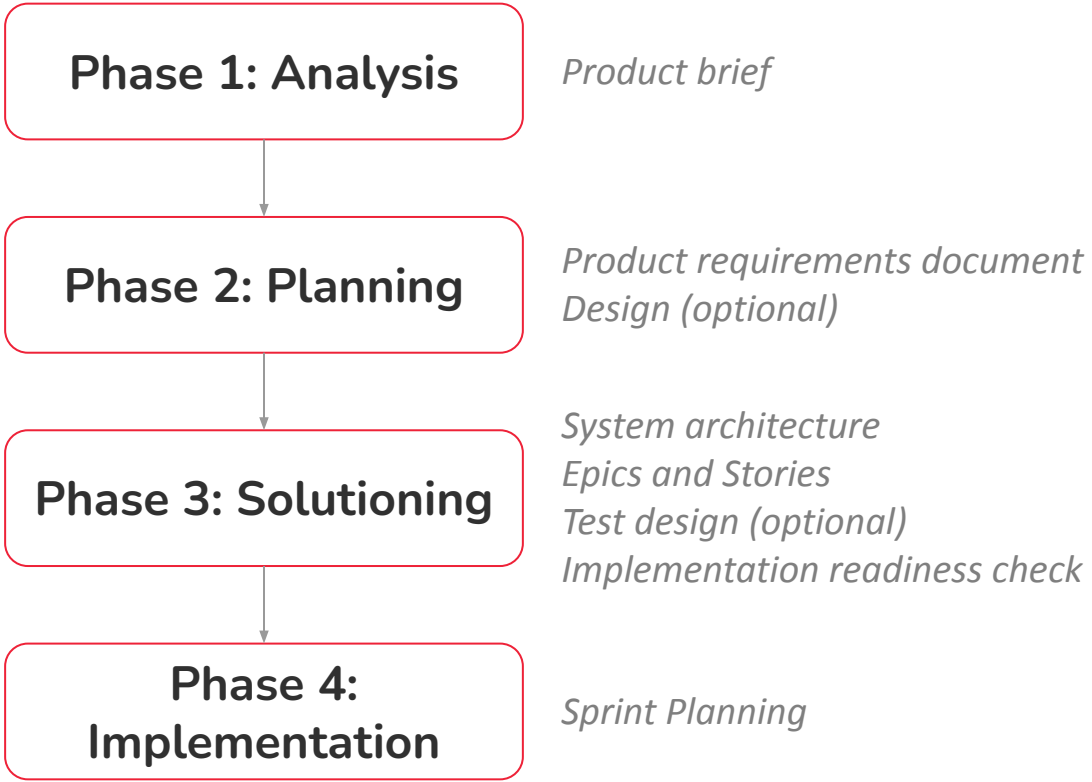
- Explain what you wish to do and choose [LW] List workflows





© ZENIKA 2026 | All rights reserved - Proprietary & Confidential

BMad Standard Flow



© ZENIKA 2026 | All rights reserved - Proprietary & Confidential